

Inheritance — Building New Classes from Old Ones

Review worksheet — base/derived classes, `protected` access, constructor chaining, overriding — builds on Worksheets 01 & 04

Name _____ Date _____ Section _____

part 1 What Is Inheritance?

Inheritance lets one class reuse the data members and methods of another, then add or change things in a small way. The class being reused from is the **base class** (a.k.a. parent / superclass); the class doing the reusing is the **derived class** (a.k.a. child / subclass). We say a derived class “**is-a**” base class.

```
class Vehicle {
protected:
    string make;
    string model;
    int topSpeed;
public:
    Vehicle(string mk, string mo, int spd)
        : make(mk), model(mo), topSpeed(spd) {}

    void describe() {
        cout << make << " " << model
            << " (" << topSpeed << " mph)\n";
    }
};

class Car : public Vehicle {
    int numDoors;
public:
    Car(string mk, string mo, int spd, int doors)
        : Vehicle(mk, mo, spd), numDoors(doors) {}
};
```

Every `Car` automatically has `make`, `model`, `topSpeed`, and `describe()` — nobody retyped them inside `Car`.

QUESTION 1

FILL IN THE BLANKS

1. `Vehicle` is the _____ class; `Car` is the _____ class.
2. The relationship between them is best described as “a `Car` _____ a `Vehicle`.”
3. The colon in `class Car : public Vehicle` means `Car` _____ from `Vehicle`.

QUESTION 2

CIRCLE EVERY MEMBER EVERY CAR OBJECT HAS

Even though none of these are declared inside `Car`, circle every one that still exists as part of `Car c("Honda", "Civic", 120, 4);`, purely because `Car` inherits from `Vehicle`:

`make` `model` `topSpeed` `describe()`

(Whether `main()` is *allowed to reach* each one directly is a separate question — that's Part 2.)

Inheritance — Building New Classes from Old Ones

Review worksheet — base/derived classes, `protected` access, constructor chaining, overriding — builds on Worksheets 01 & 04

part 2

Access Control in a Hierarchy

builds on W01

You already know `private` and `public` from Worksheet 01. Inheritance introduces a third level: `protected`. It behaves like `private` to the outside world, but a **derived class can still reach it**.

ACCESS SPECIFIER	VISIBLE INSIDE THE SAME CLASS	VISIBLE IN A DERIVED CLASS	VISIBLE OUTSIDE (<code>MAIN()</code>)
<code>private</code>	_____	_____	_____
<code>protected</code>	_____	_____	_____
<code>public</code>	_____	_____	_____

Fill each cell with Y or N.

QUESTION 3

COMPILES? Y / N

```
class Vehicle {
    string make;    // private (no label = private)
public:
    Vehicle(string mk) : make(mk) {}
};

class Car : public Vehicle {
public:
    Car(string mk) : Vehicle(mk) {}
    void printMake() {
        cout << make;    // line X
    }
};
```

Does line X compile? _____ Explain why, and state the one-word fix to `Vehicle` that would make it work.

Inheritance — Building New Classes from Old Ones

Review worksheet — base/derived classes, `protected` access, constructor chaining, overriding — builds on Worksheets 01 & 04

part 3 Constructor Chaining

builds on W04 initializer lists

Recall from Worksheet 04: a member initializer list runs *before* the constructor body. The same idea extends to inheritance — a derived class's initializer list can call a specific base-class constructor by name. The base part of the object is always built **first**, whether you ask for it or not.

```
// Relies on Vehicle's default
// constructor -- but Vehicle
// doesn't have one!
Car::Car(string mk, string mo,
         int spd, int doors) {
    numDoors = doors;
}
```

```
// Explicitly chains to the
// Vehicle constructor that
// matches these arguments
Car::Car(string mk, string mo,
         int spd, int doors)
    : Vehicle(mk, mo, spd),
      numDoors(doors) {}
```

QUESTION 4

CONSTRUCTION ORDER

```
class A { public: A() { cout << "A "; } };
class B : public A { public: B() { cout << "B "; } };
class C : public B { public: C() { cout << "C "; } };

int main() {
    C obj;
}
```

What does this print? _____

If each class also had a destructor that printed its own letter, in what order would `~A()`, `~B()`, `~C()` fire when `obj` goes out of scope?

QUESTION 5

WRITE THE CONSTRUCTOR

Given `Vehicle` from Part 1, write a constructor for a new class `Motorcycle : public Vehicle` that adds `bool hasSidecar;`. Chain to `Vehicle`'s constructor — do not reassign `make` / `model` / `topSpeed` in the body.

```
Motorcycle::Motorcycle(string mk, string mo, int spd, bool sidecar)
```

Inheritance — Building New Classes from Old Ones

Review worksheet — base/derived classes, `protected` access, constructor chaining, overriding — builds on Worksheets 01 & 04

part 4 Overriding vs. Overloading

builds on W02/W04 overloading

Worksheet 04 covered **overloading**: multiple functions, same name, *different parameter lists*, living in the same class. **Overriding** is a different thing entirely: a derived class redefines a method it inherited, using the *exact same signature*, to change what that method does for objects of the derived type.

```
class Vehicle {
public:
    void describe() { cout << "A vehicle.\n"; }
};
class Car : public Vehicle {
public:
    void describe() { cout << "A car.\n"; }    // overrides Vehicle::describe
};
```

QUESTION 6

PREDICT THE OUTPUT

```
Vehicle v;
Car c;
v.describe();
c.describe();
```

`v.describe();` prints: _____
`c.describe();` prints: _____

Inside `Car::describe()`, what line of code would call the *base class's* version instead of recursing into itself?

QUESTION 7 — LABEL EACH PAIR

OVERLOADED / OVERRIDDEN / NEITHER

<code>Car(string mk)</code>	and	<code>Car(string mk, int doors)</code>	_____
<code>Vehicle::describe()</code>	and	<code>Car::describe()</code> (same signature)	_____
<code>Vehicle::describe()</code>	and	<code>Car::honk()</code>	_____
<code>Fraction operator+(Fraction)</code>	and	<code>Fraction operator+(int)</code>	_____

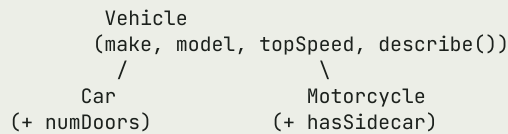
Inheritance — Building New Classes from Old Ones

Review worksheet — base/derived classes, `protected` access, constructor chaining, overriding — builds on Worksheets 01 & 04

part 5 Hierarchical Inheritance

one base, many derived

A single base class can have **several** derived classes, each specializing it differently. This is the most common shape a class hierarchy takes.



QUESTION 8 — WRITE THE CLASS

MOTORCYCLE

Write the full `Motorcycle` class declaration: it derives publicly from `Vehicle`, adds a private `bool hasSidecar;`, a constructor (use your answer from Question 5), and an **overridden** `describe()` that calls `Vehicle::describe()` first and then also prints whether it has a sidecar.

```
class Motorcycle :
```

```
};
```

Inheritance — Building New Classes from Old Ones

Review worksheet — base/derived classes, `protected` access, constructor chaining, overriding — builds on Worksheets 01 & 04

part 6 Find the Error

three traps

CASE A COMPILES, BUT...

```
class Car : Vehicle {
public:
    Car(string mk, string mo, int spd)
        : Vehicle(mk, mo, spd) {}
};

int main() {
    Car c("Kia", "Rio", 110);
    c.describe();    // line X
}
```

The missing keyword before `Vehicle` means this is **private** inheritance by default (recall: class members default private too — W01). Does line X compile? _____ Why?

CASE B COMPILES? Y / N

```
class Vehicle {
public:
    Vehicle(string mk) { make = mk; }
protected:
    string make;
};

class Car : public Vehicle {
public:
    Car(int doors) : numDoors(doors) {}
private:
    int numDoors;
};
```

Does `Car(int doors)` compile? _____ What is missing from its initializer list?

CASE C COMPILES? Y / N

```
class Vehicle {
protected:
    string make;
public:
    Vehicle(string mk) : make(mk) {}
};

int main() {
    Vehicle v("Ford");
    cout << v.make;    // line Y, called from main(), not from a derived class
}
```

Does line Y compile? _____ `protected` opened the door for derived classes — does it also open the door for `main()`? Explain.

Inheritance — Building New Classes from Old Ones

Review worksheet — base/derived classes, `protected` access, constructor chaining, overriding — builds on Worksheets 01 & 04

part 7

A Peek Ahead: Polymorphism

One more wrinkle we haven't touched yet. Suppose you store a `Car` through a `Vehicle*` pointer:

```
Vehicle* v = new Car("Kia", "Rio", 110, 4);
v->describe(); // prints "A vehicle." -- NOT "A car."
```

Even though `v` is secretly pointing at a `Car`, it calls `Vehicle::describe()` — because plain member functions are bound to the pointer's *declared* type, not what it actually points to. Marking a method `virtual` in the base class fixes this and makes the *actual* object's version run instead. That's **polymorphism**, and it's next.

cheat sheet

Inheritance at a Glance

CONCEPT	TYPICAL SYNTAX	PURPOSE
Derive publicly	<code>class Car : public Vehicle</code>	Car reuses & extends Vehicle
<code>protected</code>	<code>protected: string make;</code>	Hidden outside, visible to derived classes
Constructor chaining	<code>Car(...) : Vehicle(mk, mo, spd), numDoors(d) {}</code>	Build the base part first, explicitly
Construction order	base → derived	Base must fully exist before derived body runs
Destruction order	derived → base	Reverse of construction, always
Overriding	<code>void describe()</code> redefined in <code>Car</code>	Same signature, new behavior for the derived type
Call base version	<code>Vehicle::describe();</code>	Reach the hidden base implementation from inside an override

Overriding is not a fourth kind of constructor concept and it's not overloading — overloading picks a function by *different parameters* at compile time; overriding replaces a function with the *same signature* for objects of the derived type.